

Tracking Recruiter Performance With Unipile

A plain-English implementation guide to the Unipile-dependent metrics in the Project Beacon Recruiter Evaluation Rubric

Prepared August 10, 2026 | Sources: developer.unipile.com (linked throughout) and the Project Beacon Recruiter Evaluation Rubric

This guide explains, in simple terms, exactly how to pull real data out of Unipile for the handful of rubric metrics that actually depend on LinkedIn messaging activity — not the ones that come from your own database. Every technical claim below links straight to the Unipile page it comes from, so you (or whoever builds this) can verify it directly rather than taking this document's word for it.

0. Which of the 11 metrics actually need Unipile?

The rubric's 11 scored metrics are calculated from simple, everyday data: when a lead was assigned, when it was contacted, and what stage it's in. Only the metrics that depend on “when it was contacted” need anything from Unipile — the rest live entirely in your own pipeline database.

#	Metric	Needs Unipile?	Real data source
1	Outreach volume	YES	Unipile message history
2	Follow-up persistence	YES	Unipile message history
3	Proactive sourcing	No	Your own lead database
4	Time-to-first-touch	YES	Unipile timestamp + your assignment date
5	SLA adherence (urgent leads)	YES	Unipile timestamps, both directions
6	Backlog aging	YES	Unipile message history (or its absence)
7	Progression rate	No	Your pipeline stage field
8	Closure rate	No	Your pipeline stage field
9	Reason-logged rate	No	Your recruiter notes field
10	Interview conversion	No	Your pipeline stage field
11	Onboarding vs. queue size	No	Your pipeline stage field

So: **5 of the 11 metrics** (Outreach volume, Follow-up persistence, Time-to-first-touch, SLA adherence, and Backlog aging) need something from Unipile. The rest of this guide covers only those five, plus the shared plumbing (timestamps, sent-vs-received logic, webhooks) they all rely on.

1. The one rule that matters most: which timestamp do you trust?

Every message that passes through Unipile carries a field called `timestamp`. Per Unipile's own documentation, this is the moment the provider's server (LinkedIn, in our case) actually acknowledged the message — not the moment your own backend tried to send it. It can be used directly as the message's official “sent date.”

This distinction matters a lot for Beacon specifically. Message dispatch goes through a rate-limited queue worker (see Section 6 for why), so there can be a real gap between “recruiter approves the draft” and “message actually leaves for LinkedIn.” If you clocked “sent” using your own dispatch-call timestamp instead of Unipile's, you'd really be measuring how fast your own code fired an API call — and because the real send can lag behind a busy queue, that would make a recruiter's Time-to-first-touch look better than it actually is.

There are three candidate “first touch” instants, and they are genuinely different things:

- **Your own dispatch-call time** — when your backend fired the API request. Don't use this for KPIs.
- **Provider-acknowledged time** (Unipile's `timestamp` field) — use this one. It's the earliest point you can honestly defend as “actually sent,” not merely “attempted.”
- **Provider-delivered time** (the `delivered` field) — only supported on some providers, and even when it's true, it only confirms the message reached the provider's server, not that the recipient actually received it. Don't anchor KPIs on this either.

Rule for this whole document: every “when was it sent” and “when did they reply” calculation below uses the `timestamp` field from the Unipile message object — never your own application clock.

■ **Documentation reference:** Message object — field-by-field reference — <https://developer.unipile.com/docs/message-payload>

2. Telling “we sent it” from “they replied” apart

Every metric in this guide needs to know, for a given message, which direction it went. Unipile gives you two different ways to work this out, depending on which data source you're reading from — and it's easy to reach for the wrong one.

2.1 Reading messages through the REST API

When you fetch messages directly (for example, a chat's message history), each message object includes an `is_sender` field: true if your connected recruiter account sent it, false if the other person did. This is the simplest signal available, and it's ready to use as-is.

2.2 Reading messages through the webhook

The New messages webhook payload does **not** include an `is_sender` field. Instead, Unipile's documentation gives you the workaround directly: compare `account_info.user_id` (your own connected LinkedIn account) against `sender.attendee_provider_id` (whoever actually wrote the message). If they match, your recruiter sent it. If they don't, the lead did. This also explains a naming quirk worth knowing: the event is called `message_received` for both directions — sent messages are included in that same event type, since they can be sent by the connected user from another device or from the API itself, not only by an incoming reply.

Data source	Outbound (recruiter sent it)	Inbound (lead sent it)
-------------	------------------------------	------------------------

REST API (message object)	<code>is_sender: true</code>	<code>is_sender: false</code>
New messages webhook	<code>account_info.user_id == sender.attendee_provider_id</code>	the two values differ

■ **Documentation reference:** Message object — <https://developer.unipile.com/docs/message-payload>

■ **Documentation reference:** New messages (webhook) — <https://developer.unipile.com/docs/new-messages-webhook>

3. The five metrics, one at a time

For each metric below: what it means in plain English, exactly how Unipile supplies the underlying data, the formula in shorthand, and the exact documentation page it comes from.

Metric 1 — Outreach volume

In plain English: How many outreach messages a recruiter sent this month. A simple count.

How Unipile gives you this:

Call the account-wide `List all messages` endpoint (`GET /messages`). It accepts `account_id`, `sender_id`, `after`, `before`, `limit` and `cursor` as filters.

Set `sender_id` to the recruiter's own LinkedIn provider ID and `after`/`before` to the month's date range. The number of messages returned (paginating with the `cursor` until it comes back empty) *is* that recruiter's outreach volume — there's no need to loop through every chat one by one.

Formula: `count( messages where sender_id = recruiter AND timestamp is within the month )`

■ **Documentation reference:** List all messages (API reference) — account-wide search with `sender_id/account_id/date` filters — https://developer.unipile.com/reference/messagescontroller_listallmessages

Metric 2 — Follow-up persistence

In plain English: The average number of times a recruiter messages the same lead before that lead is marked Replied or Cold. Are they trying once and giving up, or actually following up?

How Unipile gives you this:

For a specific lead, use either `GET /chats/{chat_id}/messages` (that lead's own chat) or `List all messages for a given attendee` (`GET /chat_attendees/{sender_id}/messages`), which pulls every message tied to that person across all their chats in one call.

Filter to messages where `is_sender: true`, ordered by `timestamp`, and count how many happened before the lead's first inbound reply (or before your own system marks them Cold).

Formula: `avg( count of outbound messages per lead, up to first reply or Cold marking )`

■ **Documentation reference:** List all messages for a given attendee (API reference) — per-lead history across chats — https://developer.unipile.com/reference/chatattendeescontroller_listmessagesbyattendee

■ **Documentation reference:** Retrieving messages — `GET /chats/{chat_id}/messages` — <https://developer.unipile.com/docs/get-messages>

Metric 4 — Time-to-first-touch

In plain English: How many days pass between a lead being assigned and the recruiter's first outbound message to them.

How Unipile gives you this:

The *start* of the clock is your own “lead assigned” timestamp — Unipile has no concept of assignment, so this always comes from your database.

The *end* of the clock is the `timestamp` field on the earliest message with `is_sender: true` that Unipile has for that lead. Subtract the two and convert to days.

This is exactly the metric Section 1's timestamp rule protects: use the provider-acknowledged time, not your own dispatch-call time, or a busy queue will make this number look better than reality.

Formula: `days_between( your own lead_assigned_at, earliest Unipile message.timestamp where is_sender = true )`

■ **Documentation reference:** Message object — the timestamp field — <https://developer.unipile.com/docs/message-payload>

Metric 5 — SLA adherence (urgent leads)

In plain English: For a lead flagged urgent or high priority, did the recruiter reply within the agreed window (e.g. under 2 hours)?

How Unipile gives you this:

The *start* of the clock is the `timestamp` of the inbound message that got the lead flagged (`is_sender: false`). The “urgent” or “High Priority” flag itself is your own data — Unipile has no idea a lead is urgent, it only supplies accurate timestamps.

The *end* of the clock is the `timestamp` of the next outbound message (`is_sender: true`) on that same chat after the flagged inbound message.

Delay can come from two independent places: the recruiter juggling several leads before they open this conversation, and the dispatch queue itself (Section 6) adding lag between “approve” and “actually sent.” Measuring end-to-end with two Unipile timestamps captures both automatically, without needing to separately track “recruiter opened the conversation” or “clicked approve” as extra internal events.

Formula: `next outbound message.timestamp - flagged inbound message.timestamp`
(compare vs. SLA window)

■ **Documentation reference:** Message object — the timestamp field — <https://developer.unipile.com/docs/message-payload>

■ **Documentation reference:** New messages (webhook) — <https://developer.unipile.com/docs/new-messages-webhook>

Metric 6 — Backlog aging

In plain English: The share of assigned leads that have had zero outreach at all for 3+ business days.

How Unipile gives you this:

For every lead assigned 3+ business days ago, check whether *any* outbound message (`is_sender: true`) exists in Unipile for that lead's chat, dated after the assignment. No such message means the lead is still in backlog.

Practical shortcut: since Time-to-first-touch (Metric 4) already has you storing a “first outreach timestamp” the moment the first outbound message is recorded, Backlog aging becomes a query against your own table — assignment date is 3+ business days old AND first outreach timestamp is still null — rather than a fresh live call to Unipile every time.

If you do want a live cross-check, `List all chats` (GET `/chats`) supports an unread flag plus before/after date filters — useful as a general activity signal, though it checks chat-level activity in either direction rather than specifically “did a recruiter message go out.”

Formula: % of leads where `(today - lead_assigned_at ≥ 3 business days)` AND `(first_outreach_timestamp IS NULL)`

■ **Documentation reference:** List all chats (API reference) —
https://developer.unipile.com/reference/chatscontroller_listallchats

■ **Documentation reference:** List all messages (API reference) —
https://developer.unipile.com/reference/messagescontroller_listallmessages

4. The complete flow: from draft to reply

Here is the same story end to end, matching how it actually happens: a recruiter sends a draft message first, and a reply comes back later — sometimes with a delay, because recruiters are juggling more leads than they can look at instantly.

1 Recruiter approves a draft

This lives entirely in your app. No Unipile involvement yet.

2 Your app dispatches the message

Call `POST /chats/{chat_id}/messages` if a chat with this lead already exists, or `POST /chats` to start a new one if it's the very first message to this person. This call enters your rate-limited queue worker rather than reaching LinkedIn instantly — see Section 6 for why that queue exists at all.

3 Unipile actually sends it, and tells you when

Two ways to find out it really went out: **synchronously**, the `send` endpoint itself returns 201 Created with a message object containing `id` and `timestamp` (worth confirming the exact response shape with one real sandbox call, since the interactive docs render this example via JavaScript rather than showing the raw body). **Asynchronously** (recommended, and the more reliable path for KPIs), the New messages webhook fires event: `"message_received"` — sent messages are included in this event type too, so compare `account_info.user_id` to `sender.attendee_provider_id` to confirm it was your recruiter, not the lead.

4 Store the timestamp

Save it against the lead. This single value feeds both Time-to-first-touch (Metric 4) and “first outreach timestamp,” which retires the lead from Backlog aging (Metric 6).

5 The lead eventually replies (or doesn't)

If they do, the same webhook fires again with event: `"message_received"`. This time `account_info.user_id` will NOT match `sender.attendee_provider_id`, confirming it came from the lead. Its `timestamp` is the reply instant.

6 If this lead is flagged urgent, the SLA clock starts here

The clock's start is the inbound timestamp from Step 5. It stops the moment the next outbound message timestamp appears on this same chat (back through Steps 2–3, another recruiter reply).

7 Why a gap can appear even when the recruiter reacts fast

Two independent things can add delay between “lead replied” and “recruiter's reply lands”: (a) **recruiter-side** — they're handling several leads and haven't opened this conversation yet, and (b) **queue-side** — even after they approve a reply, it sits in the same rate-limited dispatch queue as any outbound message. Because SLA adherence is measured end-to-end using two Unipile timestamps, both sources of delay are captured automatically, with no need to separately log “recruiter opened the conversation” as its own event.

5. Two ways to actually get this data

5.1 Realtime, via webhooks (recommended)

The New messages webhook fires for every message, sent or received, in real time. Create it via `POST /webhooks` or directly from the Unipile Dashboard. Unipile offers a few webhook types in total — Account status updates, New messages, New emails, Tracking email, and New relation — and for these messaging metrics you specifically want New messages.

Retry behaviour: your endpoint needs to answer with a 200 status within 30 seconds, or Unipile will retry the delivery up to five times, with a growing delay between each attempt. Design your webhook handler to acknowledge quickly and do any heavier processing afterwards, asynchronously.

Content-type gotcha: webhooks created through the API (as opposed to the Dashboard) don't automatically send a JSON content-type header. Some downstream tools need it set explicitly via the `headers` parameter when you create the webhook.

Authentication: you can attach a custom secret header (for example `Unipile-Auth`) when you create the webhook, and check for it on every incoming request to confirm it genuinely came from Unipile.

```
curl --request POST \
  --url https://{YOUR_DSN}/api/v1/webhooks \
  --header 'X-API-KEY: {YOUR_ACCESS_TOKEN}' \
  --header 'content-type: application/json' \
  --data '{
    "request_url": "https://your-app.example.com/webhooks/unipile",
    "source": "messaging",
    "headers": [{ "key": "Unipile-Auth", "value": "yoursecretkey" }]
  }'
```

■ **Documentation reference:** Webhooks overview — <https://developer.unipile.com/docs/webhooks-2>

■ **Documentation reference:** New messages (webhook) — <https://developer.unipile.com/docs/new-messages-webhook>

5.2 Polling fallback, via a cron job

If your system genuinely can't handle webhooks, call `GET /messages` on a schedule instead. Unipile's own advice: fetch a wider time window than your cron interval, in case a disconnect caused you to miss a cycle, and de-duplicate incoming records using each message's own `id`, since the same message can show up more than once this way.

All list-style endpoints are paginated: a response includes a `cursor` value, which you feed back into the next request's `cursor` parameter to get the following page. A null cursor means there's nothing left to fetch.

■ **Documentation reference:** Retrieving messages — the cron-job fallback — <https://developer.unipile.com/docs/get-messages>

■ **Documentation reference:** API Usage — authentication and pagination — <https://developer.unipile.com/docs/api-usage>

6. Why there's a queue at all: LinkedIn's own limits

Unipile's guidance for LinkedIn automation is explicit: actions should be spaced out randomly rather than fired back-to-back, run during normal working hours, and paced especially conservatively on new or low-connection accounts — because LinkedIn's bot detection is aggressive, and a restricted company account does far more damage than slightly slower outreach. For general messaging-type routes without a specific published number, the baseline guidance is a default ceiling of about 100 actions per day per account, again spread out rather than bursted.

Why this matters for Beacon: this is exactly why a dispatch queue sits between “recruiter clicks approve” and “message actually leaves.” With 5 recruiters each attached to their own LinkedIn account and potentially hundreds of leads, this pacing is what protects those accounts from getting flagged or restricted. It's also precisely why Section 1's rule — always trust Unipile's own timestamp, never your dispatch-call time — isn't just theoretical. The gap it's protecting you from measuring incorrectly is real and expected by design.

■ **Documentation reference:** Provider Limits and Restrictions — LinkedIn pacing rules — <https://developer.unipile.com/docs/provider-limits-and-restrictions>

7. Recommended additions to your data model

- Add a `raw_provider_timestamp` column on your outreach record, populated only from the timestamp field on real Unipile message events — never from your own worker's clock.
- Compute Time-to-first-touch, SLA adherence, and Backlog aging off that one column exclusively, so all three metrics stay auditable against a single source of truth instead of drifting apart against separate internal clocks.
- Store the Unipile `message_id` alongside it, so a retried webhook delivery (Section 5.1) can't accidentally double-count an outreach message.
- Keep the sent-vs-received comparison logic (Section 2) in one shared function, since you'll need it in at least four places: Outreach volume, Follow-up persistence, Time-to-first-touch, and SLA adherence.

8. What Unipile does *not* give you

Unipile's job in this whole rubric is narrow but critical: accurate timestamps and accurate direction (sent vs. received) for every LinkedIn message. Everything about what those timestamps *mean* for a recruiter's score is built in your own application:

- **“Business days”** — Unipile timestamps are plain UTC datetimes. Deciding what counts as a business day (weekends, holidays, working hours) is entirely your own logic.
- **“Assigned”** — there's no concept of a lead being assigned to a recruiter inside Unipile. That's your own pipeline data.
- **“Flagged” / “urgent” / “High Priority”** — same story. Unipile has no idea a lead is urgent; it only gives you trustworthy timestamps for whatever you decide to measure.
- **“Replied” vs. “Cold” stage** — these are your own pipeline states, set by your own business rules (for example, after N follow-ups with no reply). Unipile just tells you whether, and precisely when, a message went in each direction.